DAY 7 | HANDS-ON ACTIVITIES

FastAPI Advanced, Security and MongoDB: Problem Statements

Nine problems building a secure SmartDesk-style API. mongomock in Colab, a real MongoDB in VS Code.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Map the Risks NEW (v4)

Level: Warm-up | **Concepts:** OWASP API Security Top 10

For six SmartDesk scenarios (for example: changing a ticket id in the URL shows another customer's ticket; limit=100000 on a list; a member calls DELETE /admin/users), name the OWASP API risk and one fix.

Sample run / expected result

```
Change id in URL      -> API1 Broken object level authorisation -> filter by owner
limit=100000          -> API4 Unrestricted resource consumption -> Query(le=50)
```

Hints

- Use the 2023 list from the deck.

Problem 2: Shared Pagination

Level: Easy | **Concepts:** Depends(), reusable dependency

Write a pagination dependency (skip >= 0, limit 1 to 50) and use it on two endpoints. Show that invalid values return 422.

Sample run / expected result

```
GET /items?skip=0&limit=5 -> {"skip": 0, "limit": 5}
GET /items?limit=500 -> 422
```

Hints

- Use Query() inside the dependency for rules.

Problem 3: Timing, Request Id and Clean Errors

Level: Easy | **Concepts:** middleware, exception handlers

Add middleware that sets X-Process-Time-ms and X-Request-ID on every response. Add a custom exception TicketLocked with a handler that returns 409 and a short message.

Sample run / expected result

```
GET /health 200 1.3 ms  X-Request-ID: 3f2a...
PATCH closed ticket -> 409 {"detail": "Ticket is closed"}
```

Hints

- import uuid; str(uuid.uuid4())
- @app.exception_handler(TicketLocked)

Problem 4: Hash and Verify

Level: Easy | **Concepts:** bcrypt or pbkdf2, salt

Write hash_password(pw) and verify_password(pw, stored). Show that the same password gives different hashes for two users.

Sample run / expected result

```
verify_password("secret1", stored) -> True
verify_password("wrong", stored) -> False
```

Hints

- bcrypt.hashpw(pw.encode(), bcrypt.gensalt())

Problem 5: Short-lived Scoped Tokens NEW (v4)

Level: Medium | **Concepts:** PyJWT, exp, scopes

Create tokens with sub, role, scope and a 15 minute expiry. Add require_scope("tickets:write"). Show a valid token works, an expired token gives 401, and a read-only token gives 403 on a write route.

Sample run / expected result

```
valid write token   -> 201
expired token       -> 401
read-only token     -> 403
```

Hints

- Create an expired token with minutes=-1.

Problem 6: Protected Tickets in MongoDB

Level: Challenging | **Concepts:** current_user, ownership, PyMongo

Add register, login and GET/POST/DELETE /tickets stored in MongoDB. Each ticket stores its owner; users see and delete only their own. Registration must not accept a role from the client.

Sample run / expected result

```
Hana sees 2 tickets, Ravi sees 1
Ravi deleting Hana's ticket -> 404
register with role=admin -> role stays customer
```

Hints

- Filter every query by {"owner": user}.
- This is API1 and API3 in practice.

Problem 7: Analytics and Pagination

Level: Medium | **Concepts:** aggregation, skip and limit

Add GET /analytics/by-priority (count of open tickets per priority, highest first) and paginate GET /tickets with page and size (size at most 50), returning items and total.

Sample run / expected result

```
[{"priority": "high", "open": 7}, {"priority": "medium", "open": 4}]
{"page": 2, "size": 5, "total": 12, ...}
```

Hints

- $match, $group, $sort, $project

Problem 8: Break It, Then Fix It: NoSQL Injection NEW (v4)

Level: Challenging | **Concepts:** NoSQL injection, input validation

Run the deliberately vulnerable login and search endpoints from the notebook. Log in as admin without the password using an operator payload, and dump all tickets with a regex payload. Then fix both with Pydantic models and re.escape, and prove the payloads now return 422 or no results.

Sample run / expected result

```
vulnerable login {"password": {"$ne": ""}} -> 200 (bad)
fixed login same payload             -> 422
fixed search q=".*"                  -> 0 results
```

Hints

- Only ever test injection on your own local code.

Problem 9: Vulnerability Report v1 NEW (v4)

Level: Stretch | **Concepts:** bandit, pip-audit, reporting

Run bandit on the code you generated with Codex today and pip-audit on its requirements. Record every finding in the vulnerability report template, fix at least two (Codex-assisted is fine), re-scan, and mark them verified with evidence.

Sample run / expected result

```
| V-01 | Hard-coded secret | bandit B105 | Low | app/security.py:4 | env var | Codex-assisted
```

Hints

- python -m bandit -r app -f json
- pip-audit -r requirements.txt